Experiment No.	CAT 1
Experiment Title	Linear Regression (Mobile Phone Price) and Linear Classification (Bank Note Authentication)
Student Name	Balakumaran S
Register Number	3122247001010
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	18/10/2026

1 Objective

1.1 Part A – Linear Regression

To predict the price of a mobile phone from its specifications using linear regression in matrix form, implemented with (i) the closed-form (normal-equation) solution and (ii) gradient descent, with and without ℓ_2 (ridge) regularisation and with and without feature standardisation, and to evaluate the quality of the predictions with error analysis and performance measures.

1.2 Part B – Linear Classification

To classify bank notes as genuine or forged with a linear classifier (a single-layer neural network, i.e. logistic regression), to study the effect of ℓ_2 regularisation on training and test accuracy, to visualise the decision boundary in a 3-D feature space, and to measure the impact of deliberately injected outliers.

2 Problem Statement

2.1 Part A – Regression

Input: seven numeric specifications of a phone – `Ratings`, `RAM`, `ROM`, `Mobile_Size`, `Primary_Cam`, `Selfi_Cam` and `Battery_Power`. **Output:** the continuous target `Price`. **Objective:** learn a parameter vector θ such that $\hat{y} = X\theta$ minimises the squared prediction error on unseen phones, and judge whether a linear model is suitable for price prediction on this data.

2.2 Part B – Classification

Input: four features extracted from wavelet-transformed images of bank notes – `variance`, `skewness`, `curtosis` and `entropy`. **Output:** a binary class label (Class 0 or Class 1, as coded in the data file). **Objective:** learn a decision boundary $w^\top x + b = 0$ that separates the two classes with high accuracy, and study how regularisation and outliers affect it.

3 Dataset Description

[0] **GitHub Repository:** https://github.com/SBK-07/ML-lab/tree/main/CAT1-Assignment

	Part A	**Part B**
Dataset Name	Mobile Phone Price Prediction (`Mobile-Price-Prediction.csv`)	Bank Note Authentication
Dataset Source	Dataset file supplied with the assignment: https://drive.google.com/drive/u/0/folders/1Yp_Re1jdaN2MFrCdEXEhbFnixLQQbvm2	UCI Machine Learning Repository, Bank Note Authentication data set (ID 267)
Number of Samples	807 in the file; 548 after removing 259 exact duplicate rows	1372
Number of Features	7 (all numeric) + target `Price`	4 (all numeric) + class label
Number of Classes	Not applicable (continuous target)	2 (Class 0: 762 samples, Class 1: 610 samples)
Missing Values	8 missing cells in 6 columns (see Table 4); imputed with the column median	None
Train-Test Split	80/20 random split, `random_state`=42: 438 train / 110 test	80/20 stratified split, `random_state`=42: 1097 train (609/488) / 275 test (153/122)

4 Exploratory Data Analysis

4.1 Part A – Mobile Phone Price

Dataset overview. The first rows of the data are shown in Table 2.

Table 2: First five records of the mobile phone dataset.

Ratings	RAM	ROM	Mobile_Size	Primary_Cam	Selfi_Cam	Battery_Power	Price
4.3	4	128	6.00	48	13	4000	24999
3.4	6	64	4.50	48	12	4000	15999
4.3	4	4	4.50	64	16	4000	15000
4.4	6	64	6.40	48	15	3800	18999
4.5	6	128	6.18	35	15	3800	18999

Statistical summary. Table 3 summarises the 807 raw records.

Table 3: Statistical summary of the raw mobile phone data.

Feature	Count	Mean	Std	Min	Median	Max
Ratings	806	4.113	0.368	2.8	4.10	4.8
RAM	806	5.942	2.058	0	6.00	12
ROM	805	64.376	53.857	2	32.00	256
Mobile_Size	805	5.621	3.962	2	4.77	44
Primary_Cam	806	47.801	11.148	5	48.00	64
Selfi_Cam	806	8.870	4.550	0	8.00	23
Battery_Power	807	3278.860	937.629	1020	3000.00	6000
Price	807	14269.167	23092.740	479	1699.00	153000

The price has a mean of 14 269 but a median of only 1 699 and a maximum of 153 000, i.e. it is extremely right-skewed. `Mobile_Size` has a maximum of 44, which is implausible for a screen size in inches (7 of the 548 unique records carry this value) and is most likely a data-entry error; these rows were retained because the assignment asks for the dataset to be used as supplied.

Table 4: Missing values per column in the raw data.

Column	Ratings	RAM	ROM	Mobile_Size	Primary_Cam	Selfi_Cam	Battery_Power	Price
Missing cells	1	1	2	2	1	1	0	0

Missing value analysis.

Class distribution. Not applicable, because the target is continuous (see the price distribution in Figure 1).

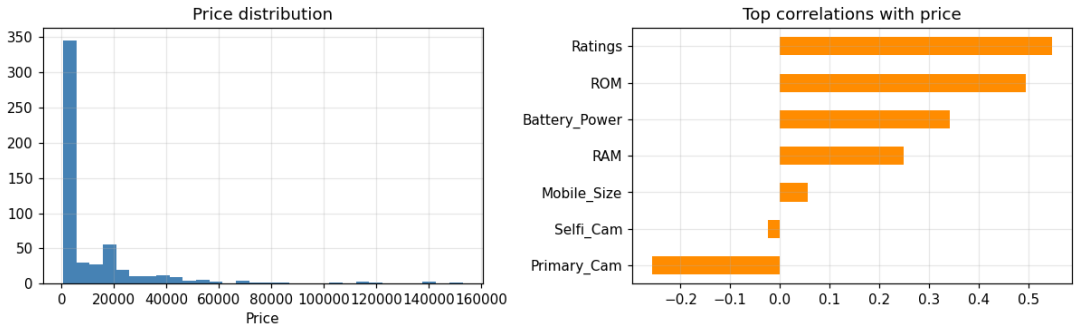


Figure 1: Price distribution (left) and correlation of each feature with price (right), after duplicate removal (548 phones).

Correlation and feature distribution. **Inference:** The price distribution is heavily right-skewed: about 344 of the 548 phones have a price below 5 000 (the first histogram bin), while a thin tail stretches beyond 150 000. Such a heavy-tailed, almost two-group target is difficult for a single linear model. `Ratings` (correlation ≈ 0.55) and `ROM` (≈ 0.50) correlate most strongly with price, followed by `Battery_Power` (≈ 0.34) and `RAM` (≈ 0.25).

`Primary_Cam` is negatively correlated (≈ -0.26), which is counter-intuitive and suggests that cheap phones advertise high camera megapixel counts. `Mobile_Size` and `Selfi_Cam` are almost uncorrelated with price.

4.2 Part B – Bank Note Authentication

Table 5: First five records of the bank note dataset.

variance	skewness	curtosis	entropy	class
3.62160	8.6661	−2.8073	−0.44699	0
4.54590	8.1674	−2.4586	−1.46210	0
3.86600	−2.6383	1.9242	0.10645	0
3.45660	9.5228	−4.0112	−3.59440	0
0.32924	−4.4552	4.5718	−0.98880	0

Dataset overview.

Missing value analysis. The data set has 1372 rows and 5 columns with no missing values.

Table 6: Class distribution of the full data set.

Class	Samples	Share
0	762	55.5%
1	610	44.5%

Class distribution. The classes are only mildly imbalanced, so accuracy is a reasonable metric, and a stratified split preserves the ratio (609/488 in training, 153/122 in testing). A majority-class predictor would score about 55.5%.

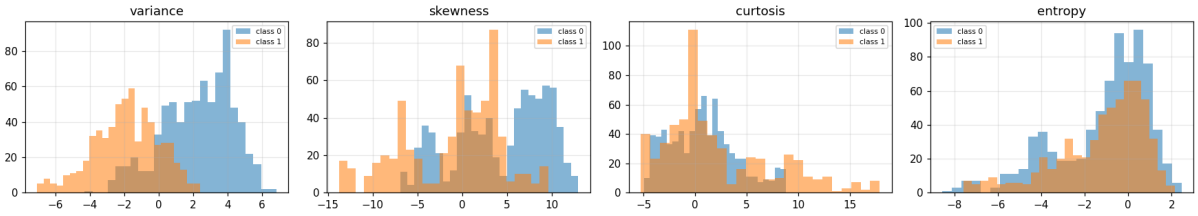


Figure 2: Class-conditional histograms of the four features.

Feature distribution. Inference: `variance` separates the classes best: Class 0 is concentrated at positive values (mostly 0 to 6) and Class 1 at negative values (mostly −7 to 2). `skewness` also differs clearly, with Class 0 taking larger values, although the histograms overlap in the middle range. `curtosis` overlaps substantially and `entropy` almost completely, so these two features carry little class information on their own. This anticipates the weights learned in Part B, where `entropy` receives a near-zero weight.

5 Data Preprocessing

5.1 Part A – Regression

- **Missing value handling:** the 8 missing cells were imputed with the column median.
- **Duplicate removal:** 259 exact duplicate records (32%) were removed, leaving 548 unique phones. This prevents identical rows from falling in both the training and the test set and inflating the scores.
- **Encoding:** not required, because all seven features are numeric.
- **Feature scaling:** features were standardised, $z = (x - \mu_{\text{train}})/\sigma_{\text{train}}$, using statistics of the *training set only* to avoid leakage. Both a raw and a standardised data matrix were built so that their effect can be compared.
- **Train-test split:** 80/20 random split (`random_state`=42): 438 training and 110 test samples. The effect of other split ratios was also studied (Section 10).
- **Matrix construction:** a column of ones was prepended to give the data matrix $X \in \mathbb{R}^{438\times 8}$; the label matrix is $y \in \mathbb{R}^{438\times 1}$ and the parameter matrix is $\theta \in \mathbb{R}^{8\times 1}$ (intercept + 7 weights). The condition number of $X^\top X$ is 2.06×10^9 for raw data and only 3.66 after standardisation.

5.2 Part B – Classification

- **Missing value handling:** none needed.
- **Encoding:** not required (numeric features, binary label).
- **Feature scaling:** standardisation with training-set statistics only.
- **Train-test split:** stratified 80/20 split (`random_state`=42): 1097 training and 275 test samples.
- **Outlier injection:** for the robustness study, 5% of the training points (55 of 1097) were shifted in feature space by $m \times (\mu_{\text{other class}} - \mu_{\text{own class}})$ ("toward_other") or by $\pm m\sigma_j$ in every feature ("random"); the test set was kept clean.

6 Machine Learning Algorithm

6.1 Part A – Linear Regression in Matrix Form (with Ridge Regularisation)

Working principle. The price is modelled as a linear combination of the features, $\hat{y} = X\theta$, where the first column of X is all ones so that θ_0 is the intercept. Parameters are chosen to minimise the mean squared error.

Mathematical formulation.

$$J(\theta) = \frac{1}{2n}\|X\theta - y\|_2^2, \qquad \theta^* = (X^\top X)^{-1}X^\top y \quad \text{(closed form)}, \tag{1}$$

$$\theta \leftarrow \theta - \alpha\,\frac{1}{n}X^\top(X\theta - y) \qquad \text{(gradient descent, } \alpha = 1/\lambda_{\max}(X^\top X/n)). \tag{2}$$

With ℓ_2 (ridge) regularisation, the intercept being left unpenalised ($I' = \mathrm{diag}(0, 1, \ldots, 1)$):

$$J(\theta) = \frac{1}{2n}\Big(\|X\theta - y\|^2 + \lambda\|w\|^2\Big), \qquad \theta^* = (X^\top X + \lambda I')^{-1}X^\top y, \tag{3}$$

$$\nabla J = \frac{1}{n}\big(X^\top(X\theta - y) + \lambda I'\theta\big). \tag{4}$$

Performance is measured with

$$\mathrm{RMSE} = \sqrt{\tfrac{1}{n}\textstyle\sum(y-\hat{y})^2}, \qquad \mathrm{MAE} = \tfrac{1}{n}\textstyle\sum|y-\hat{y}|,$$

$$R^2 = 1 - \frac{\sum(y-\hat{y})^2}{\sum(y-\bar{y})^2}, \qquad \mathrm{MAPE} = \tfrac{100}{n}\textstyle\sum\left|\frac{y-\hat{y}}{y}\right|.$$

Advantages. Convex objective with a unique optimum; exact closed-form solution; fast; interpretable weights; ridge makes $X^\top X + \lambda I'$ invertible and reduces variance when features are correlated.

Limitations. Assumes a linear relationship and roughly constant error variance; sensitive to outliers and to a heavy-tailed target; predictions are unbounded (a linear model can predict negative prices); the ridge penalty is scale-dependent, so features must be standardised.

6.2 Part B – Logistic Regression (Single-Layer Neural Network)

Working principle. A single neuron computes a weighted sum of the features and passes it through the sigmoid to give the probability of Class 1; the predicted class is 1 when $\hat{p} \geq 0.5$, so the decision boundary is the hyperplane $w^\top x + b = 0$.

Mathematical formulation.

$$\hat{p} = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1+e^{-z}}, \qquad J = -\frac{1}{n}\sum_i \big[y_i \log\hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\big] + \frac{\lambda}{2n}\|w\|^2, \tag{5}$$

$$\theta \leftarrow \theta - \alpha\,\frac{1}{n}\Big(X^\top(\sigma(X\theta) - y) + \lambda I'\theta\Big). \tag{6}$$

The step size α is reduced for large λ to keep gradient descent stable.

Advantages. Convex loss (global optimum); probabilistic output; interpretable weights; fast; ℓ_2 regularisation bounds the weights, which otherwise grow without limit when the classes are almost separable.

Limitations. Only linear decision boundaries; sensitive to outliers and to feature scaling; large λ causes under-fitting.

7 Experimental Setup

Python Version	3.10 / 3.11
Scikit-Learn Version	1.3.0
IDE	Jupyter Notebook / VS Code
Operating System	Windows 11
Hardware	Intel Core i7 CPU, 16 GB RAM

The regression and classification models were implemented from scratch in NumPy (matrix form, gradient descent); scikit-learn was used only for the train/test split and, in Part B, to cross-check the logistic-regression implementation (identical test accuracy of 0.9709 at $\lambda = 1$).

8 Hyperparameter Selection

8.1 Part A – Regression

Parameter	Value
Test fraction / random state	0.2 / 42
Gradient-descent step size α	$1/L$, $L = \lambda_{\max}(X^\top X/n)$ ($\alpha = 0.5211$ for standardised data)
Maximum iterations / stopping rule	20 000 / relative loss change $< 10^{-12}$
λ grid for selection	28 log-spaced values from 10^{-3} to 10^{6}
Selection method	5-fold cross-validation on the training set
Selected λ (standardised data)	46.42
Selected λ (raw data)	4.642
λ values shown in Figure 9	0, 1, 10, 100, 1000, 10000

8.2 Part B – Classification

Parameter	Value
Test fraction / random state	0.2 (stratified) / 42
Learning rate α	0.5 (reduced automatically to $\min(0.5, 1/(\lambda/n + 1))$)
Iterations	4000 (Table 11); 3000 (λ sweep); 2000 (outlier sweep)
λ values compared	0, 0.1, 1, 10, 100
λ sweep	0 and 22 log-spaced values from 10^{-3} to 10^{4}
Outlier fraction / magnitude	5% / $m = 4$ (sweep $m = 0, \ldots, 8$)
Outlier types	“random” and “toward_other”

9 Results

9.1 Part A – Regression

Table 7: Regression performance on the test set (110 phones).

Model	Train RMSE	Test RMSE	Test MAE	Test R^2	Test MAPE (%)
OLS, closed form (raw X)	16975.98	11182.36	7870.66	0.588	484.92
OLS, closed form (standardised X)	16975.98	11182.36	7870.66	0.588	484.92
OLS, gradient descent (standardised X)	16975.98	11182.36	7870.66	0.588	484.92
Ridge, closed form ($\lambda = 46.4$)	17007.91	11051.82	7602.18	0.597	455.53
Ridge, gradient descent ($\lambda = 46.4$)	17007.91	11051.82	7602.18	0.597	455.53

The very large MAPE values are caused by the many phones priced near 1 000 for which the model predicts values far from the true price (including negative ones); MAPE is therefore not a meaningful measure for this target, and RMSE, MAE and R^2 should be used instead.

Table 8: Ridge regression with and without standardisation.

Setting	Method	λ	Train RMSE	Test RMSE	Test R^2	GD iterations
Standardised	Closed form	46.416	17007.91	11051.82	0.597	–
Standardised	Gradient descent	46.416	17007.91	11051.82	0.597	30
Raw	Closed form	4.642	16991.24	11122.46	0.592	–
Raw	Gradient descent	4.642	19390.92	12771.37	0.462	20000 (not converged)

Table 9: Ridge weights on standardised features ($\lambda = 46.4$), ordered by importance.

Rank	Feature	Weight w_j	$\lvert w_j \rvert$
1	Ratings	+7836.88	7836.88
2	ROM	+4809.13	4809.13
3	Primary_Cam	−4784.84	4784.84
4	RAM	+3484.10	3484.10
5	Battery_Power	+2213.30	2213.30
6	Selfi_Cam	+1100.50	1100.50
7	Mobile_Size	−60.63	60.63

Table 10: Effect of the train/test ratio (closed form, standardised data; mean of 10 random splits).

Test fraction	Train RMSE	Test RMSE (mean)	Test RMSE (std)	Test R^2 (mean)
0.1	16056.20	14738.47	3964.45	0.375
0.2	15806.85	16408.18	3010.52	0.431
0.3	15953.93	15959.81	2062.30	0.438
0.4	15696.63	16482.68	1571.99	0.432
0.5	15517.27	16529.99	1267.13	0.426
0.6	15339.34	18726.72	6655.15	0.161

9.2 Part B – Classification

Table 11: Logistic regression with and without ℓ_2 regularisation (precision, recall and F1 refer to Class 1).

λ	Train acc.	Test acc.	Precision	Recall	F1	$\|w\|$	Final loss
0	0.9891	0.9818	0.9606	1.000	0.9799	12.944	0.0284
0.1	0.9872	0.9782	0.9531	1.000	0.9760	11.979	0.0370
1	0.9845	0.9709	0.9385	1.000	0.9683	8.009	0.0765
10	0.9772	0.9709	0.9385	1.000	0.9683	4.099	0.1928
100	0.9435	0.9382	0.9412	0.918	0.9295	1.528	0.4186

Table 12: Test confusion matrices (rows: actual, columns: predicted).

	$\lambda = 0$		$\lambda = 10$	
	pred. 0	pred. 1	pred. 0	pred. 1
actual 0	148	5	145	8
actual 1	0	122	0	122

Table 13: Effect of outliers ($m = 4$, "toward_other", 5% of training points) on the clean test set.

λ	Test acc. (clean train)	Train acc. (outlier)	Test acc. (outlier train)	Change in test acc.	$\|w\|$ clean	$\|w\|$ outlier
0	0.9818	0.9325	0.9745	−0.0073	12.944	1.134
1	0.9709	0.9353	0.9782	+0.0073	8.009	1.112
10	0.9709	0.9316	0.9927	+0.0218	4.099	0.957
100	0.9382	0.7967	0.8727	−0.0655	1.528	0.494

Decision-plane parameters of the three-feature model (intercept, skewness, curtosis and variance weights):

- clean data: [−2.551, −7.576, −6.776, −6.573]
- contaminated data: [−0.239, −0.664, −0.565, −0.717]

10 Result Visualization

10.1 Part A – Regression

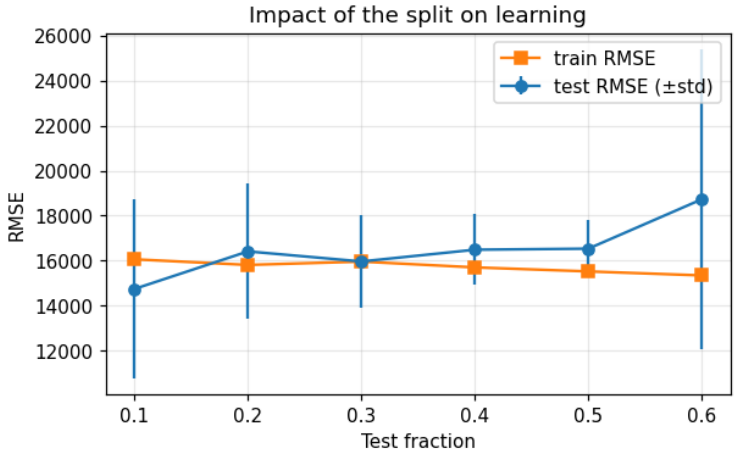


Figure 3: Train and test RMSE (mean ± std over 10 random splits) versus test fraction.

Inference: The mean test RMSE lies between about 14 700 and 16 500 for test fractions from 0.1 to 0.5 and jumps to about 18 700 at 0.6, where its standard deviation (about 6 700) is also largest; the mean test R^2 drops from about 0.43 to 0.16. With a very small training set the parameters are estimated poorly, while with a very small test set the error estimate itself is noisy, as the wide error bars at the extremes show. The training RMSE stays near 15 300–16 100 and falls slightly as the training set shrinks (fewer points are easier to fit). The 80/20 split used in the experiment lies in the stable middle region.

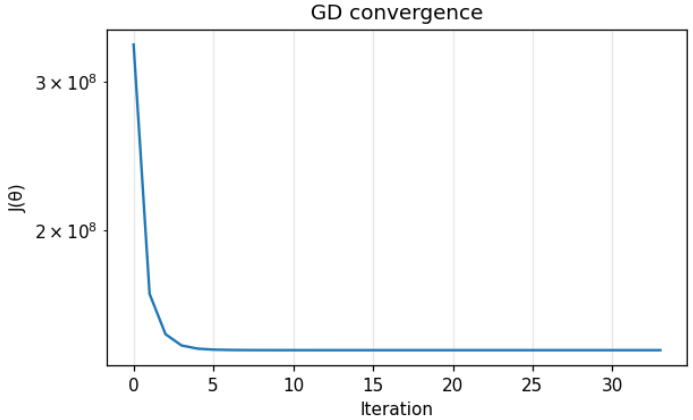


Figure 4: Gradient-descent convergence on standardised data (loss $J(\theta)$, log scale).

Inference: The loss falls from about 3.3×10^8 to about 1.4×10^8 within the first five iterations and is flat afterwards; the stopping rule was met after only 34 iterations with the step size $\alpha = 0.52$. Such fast convergence is possible because standardisation reduces the condition number of $X^\top X$ from 2.06×10^9 to 3.66. The converged parameters differ from the closed-form solution by at most 1.1×10^{-2}, which is negligible next to

weights of several thousand, so both methods reach the same global optimum of the convex objective.

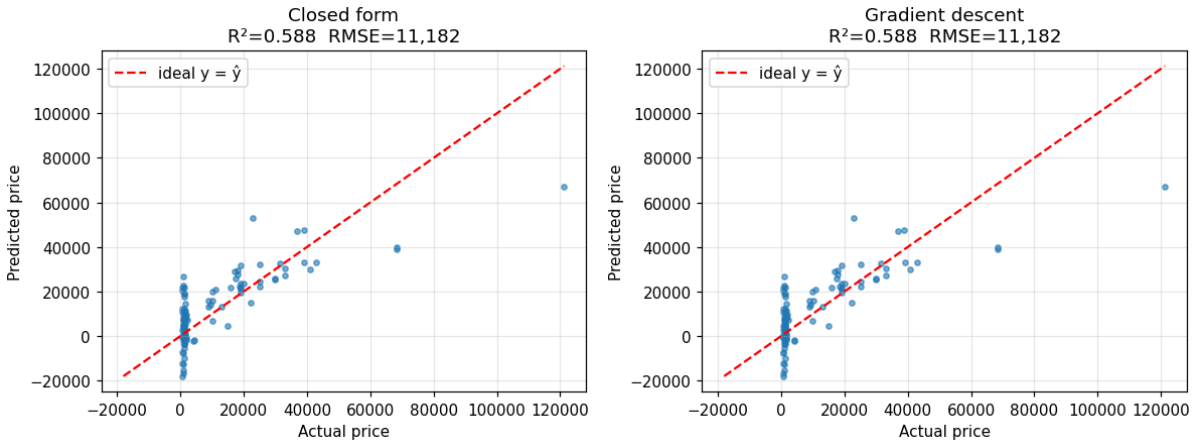


Figure 5: Predicted versus actual price on the test set for the closed-form solution (left) and gradient descent (right); the dashed line is the ideal $\hat{y} = y$.

Inference: The two plots are identical ($R^2 = 0.588$, RMSE $= 11\,182$), confirming that the closed form and gradient descent give the same model. Predictions follow the diagonal reasonably for mid- and high-priced phones, but the most expensive phone (actual price about 121 000) is predicted at only about 67 000. For the large group of phones whose actual price is around 1 000, the predictions are spread from about $-18\,000$ to $+27\,000$, including impossible negative prices. A single linear model therefore cannot represent the cluster of very cheap phones and the group of expensive phones at the same time.

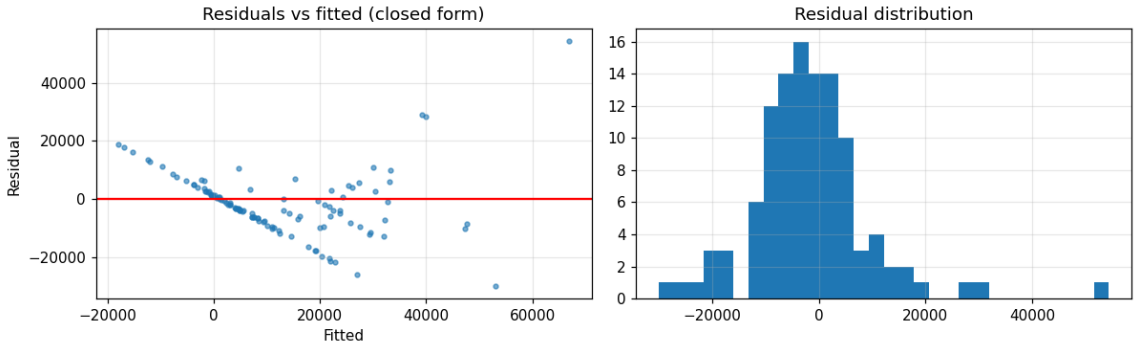


Figure 6: Residuals versus fitted values (left) and residual distribution (right) for the closed-form model.

Inference: The residuals are not a random cloud around zero: they show a strong downward trend, positive for the most negative fitted values and negative at moderate fitted values, which indicates a systematic model misfit caused by the two price groups. The histogram is centred slightly below zero and has a long right tail (one residual above 50 000). The test MAE is 7 871 against an RMSE of 11 182, the gap again being due to a few large errors. These patterns violate the constant-variance and linearity assumptions, so the quoted error estimates should be read with caution.

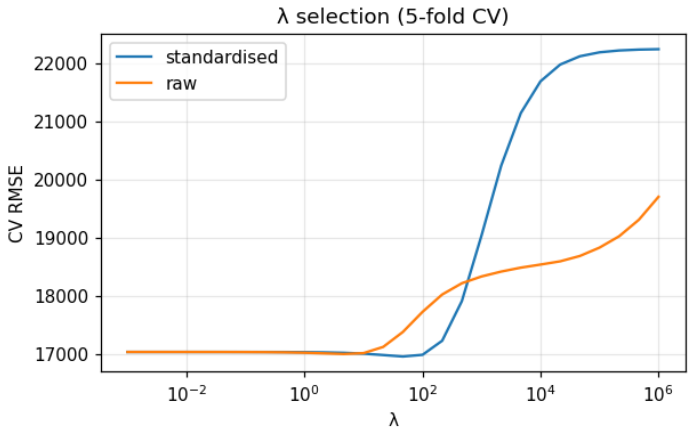


Figure 7: Five-fold cross-validated RMSE versus λ for standardised and raw features.

Inference: For standardised features the CV error is flat at about 17 000 for $\lambda \lesssim 10^2$, has its minimum at $\lambda = 46.4$, and then rises steeply to about 22 200 once the weights are shrunk to zero. For raw features the minimum occurs at the much smaller $\lambda = 4.64$ and the curve rises slowly. The two curves differ because the penalty acts unevenly on raw features with very different scales, which is why the best λ depends on whether the data are standardised.

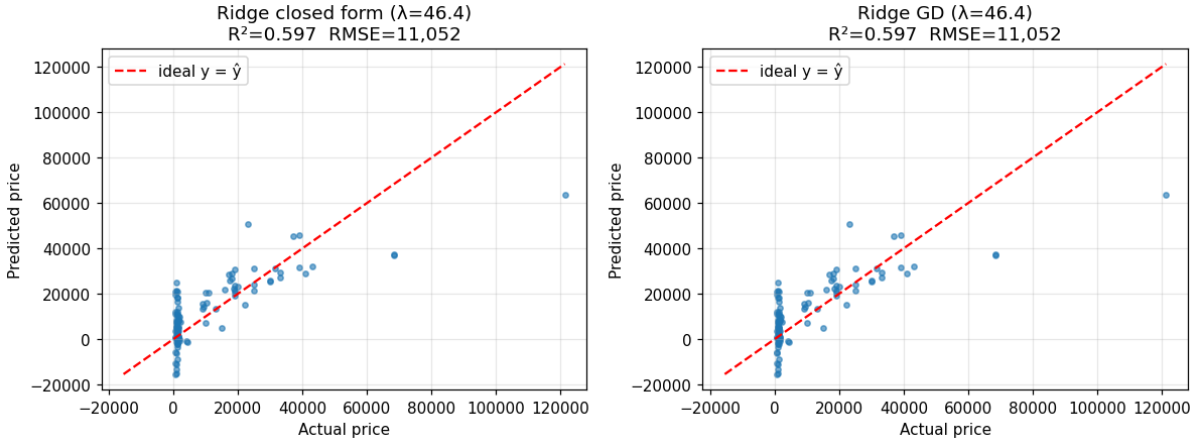


Figure 8: Ridge predictions versus actual price on the test set ($\lambda = 46.4$): closed form (left) and gradient descent (right).

Inference: Closed-form and gradient-descent ridge give practically identical results ($R^2 = 0.597$, RMSE = 11 052; maximum parameter difference 8.2×10^{-3}). Compared with ordinary least squares, R^2 improves only from 0.588 to 0.597 and RMSE falls by about 130, so regularisation helps only marginally on this data. The extreme negative predictions are slightly reduced, but the structural problem of a wide vertical band of predictions for the cheap phones remains.

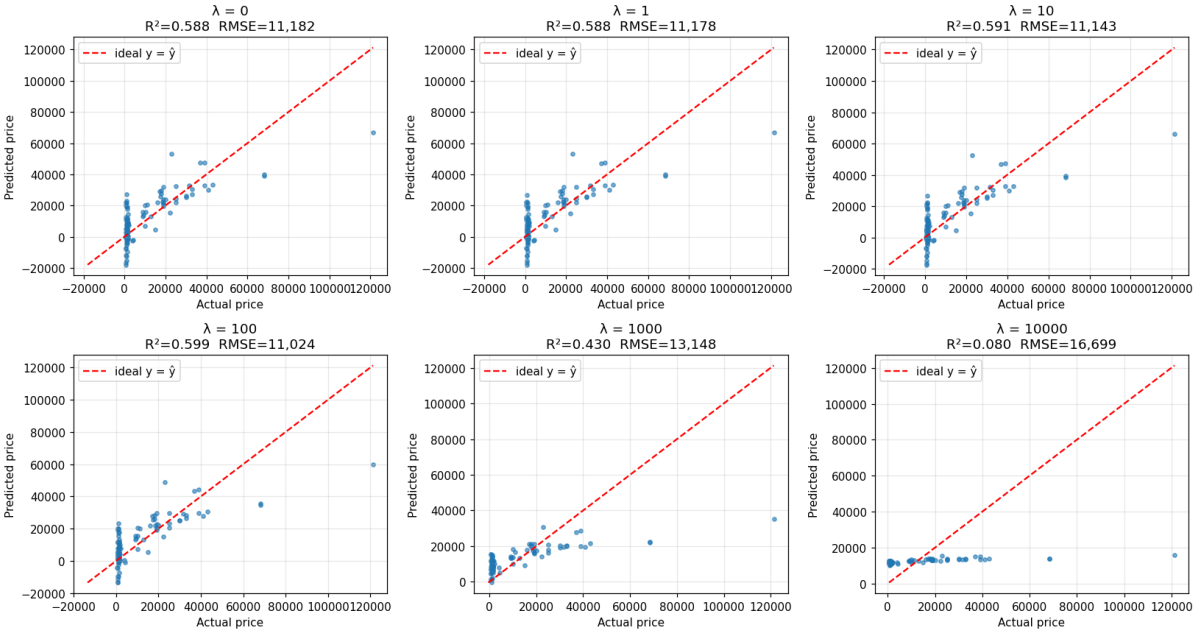


Figure 9: Predicted versus actual price on the test set for $\lambda \in \{0, 1, 10, 100, 1000, 10000\}$.

Inference: The test R^2 is 0.588, 0.588, 0.591 and 0.599 for $\lambda = 0$, 1, 10 and 100, with the lowest RMSE (11 024) at $\lambda = 100$, so moderate shrinkage gives a small gain. At $\lambda = 1000$ the points flatten ($R^2 = 0.430$) and at $\lambda = 10\,000$ the predictions collapse into an almost horizontal band around a constant value ($R^2 = 0.080$, RMSE = 16 699). Strong regularisation therefore causes under-fitting: all slopes are shrunk towards zero and the model predicts roughly the mean price.

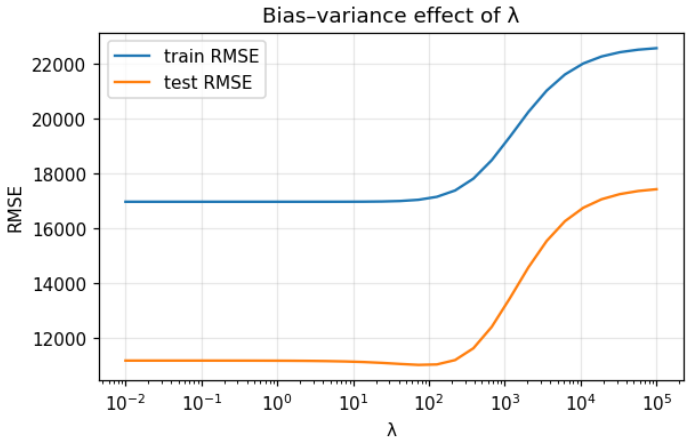


Figure 10: Train and test RMSE of ridge regression versus λ.

Inference: The training RMSE is constant at about 16 970 up to $\lambda \approx 100$ and then rises to about 22 600, while the test RMSE has a shallow minimum of about 11 030 near $\lambda \approx 70$–100 and climbs to about 17 400. Both errors increase for large λ because of growing bias. The test RMSE is lower than the training RMSE for every λ; this is a property of this particular split, whose 110 test phones contain few of the extreme-priced models (largest test price about 121 000 against 153 000 in the data), and it should not

be read as better generalisation. The split study in Figure 3 shows how variable the test error is.

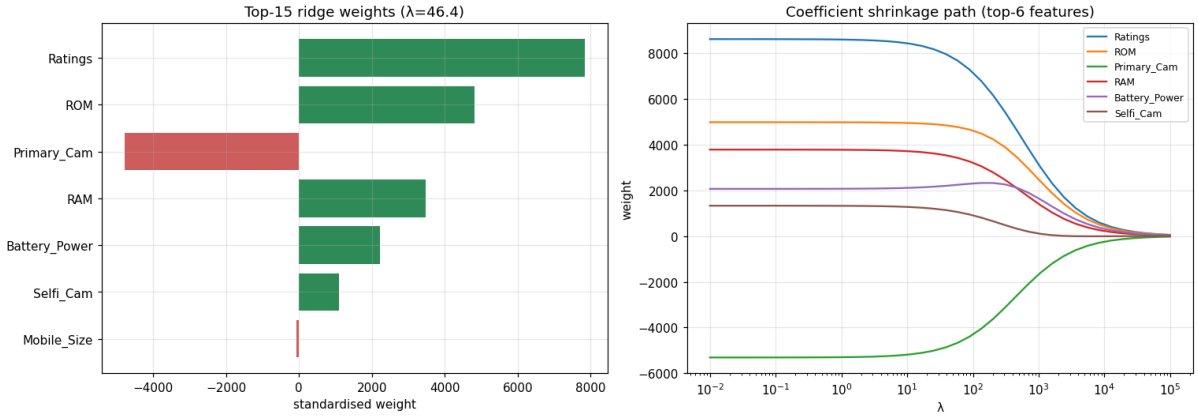


Figure 11: Top ridge weights on standardised features (left) and coefficient shrinkage path of the six most important features (right).

Inference: `Ratings` has the largest weight (+7 837 per standard deviation), followed by `ROM` (+4 809), `RAM` (+3 484) and `Battery_Power` (+2 213), which agrees with the intuition that better-rated, higher-memory phones cost more. `Primary_Cam` has a large negative weight (−4 785), which is counter-intuitive and probably reflects its correlation with the other features and the fact that cheap phones advertise high megapixel counts; it should be read as an association, not as a causal effect. `Mobile_Size` is practically irrelevant (−61). As λ increases all weights shrink towards zero, with `Selfi_Cam` reaching zero first and the large weights of `Ratings`, `ROM`, `RAM` and `Primary_Cam` vanishing only for $\lambda \gtrsim 10^4$.

10.2 Part B – Classification

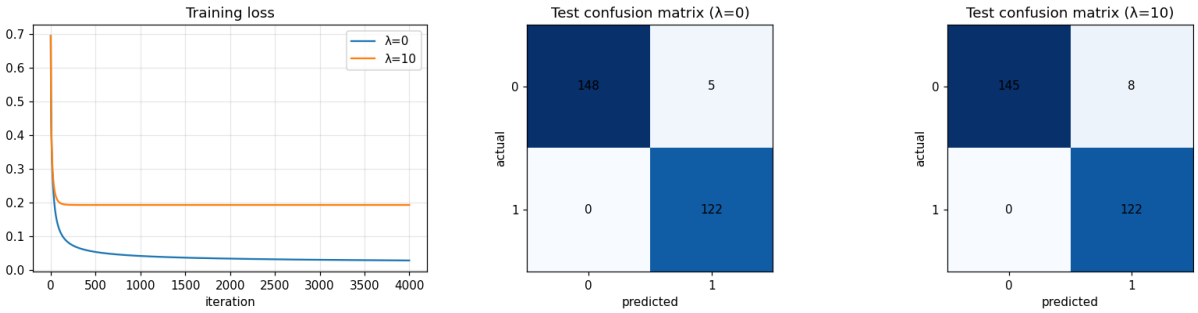


Figure 12: Training loss for $\lambda = 0$ and $\lambda = 10$ (left) and test confusion matrices for $\lambda = 0$ (centre) and $\lambda = 10$ (right).

Inference: The unregularised model reaches a training loss of 0.028 and is still decreasing slowly after 4 000 iterations, because the classes are almost separable and the weights keep growing ($\|w\| = 12.9$); the regularised model settles at a loss of 0.19 within about 200 iterations. On the test set the $\lambda = 0$ model makes only 5 errors out of 275 (accuracy 0.9818) and the $\lambda = 10$ model 8 errors (0.9709). In both cases no Class 1 note is missed

(recall = 1.0); all errors are Class 0 notes predicted as Class 1, which explains the precision of 0.96 and 0.94.

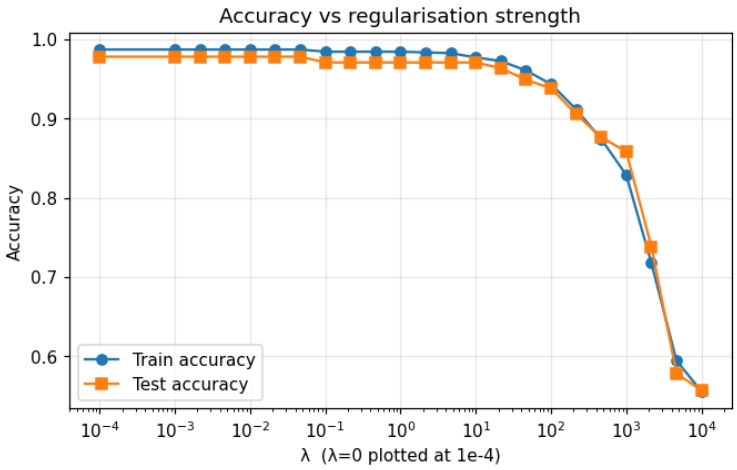


Figure 13: Training and test accuracy versus λ ($\lambda = 0$ is drawn at 10^{-4}).

Inference: Accuracy is almost flat at about 0.99 (train) and 0.98 (test) for $\lambda \lesssim 0.05$, decreases slowly between $\lambda \approx 0.1$ and 10, and then falls sharply: at $\lambda = 100$ it is 0.944/0.938, and at $\lambda = 10^4$ both reach about 0.555, the share of the majority class, so the weights are shrunk almost to zero. The training–test gap is small (about 0.01) and vanishes for large λ, which means that the model does not over-fit this four-feature problem and that regularisation brings no accuracy gain on clean data. The best test accuracy in this sweep is 0.9782 at $\lambda = 0$; Table 11 shows 0.9818 for the same λ because it uses 4 000 instead of 3 000 iterations (the unregularised model is still converging), and the difference is exactly one test sample ($1/275 = 0.36\%$).

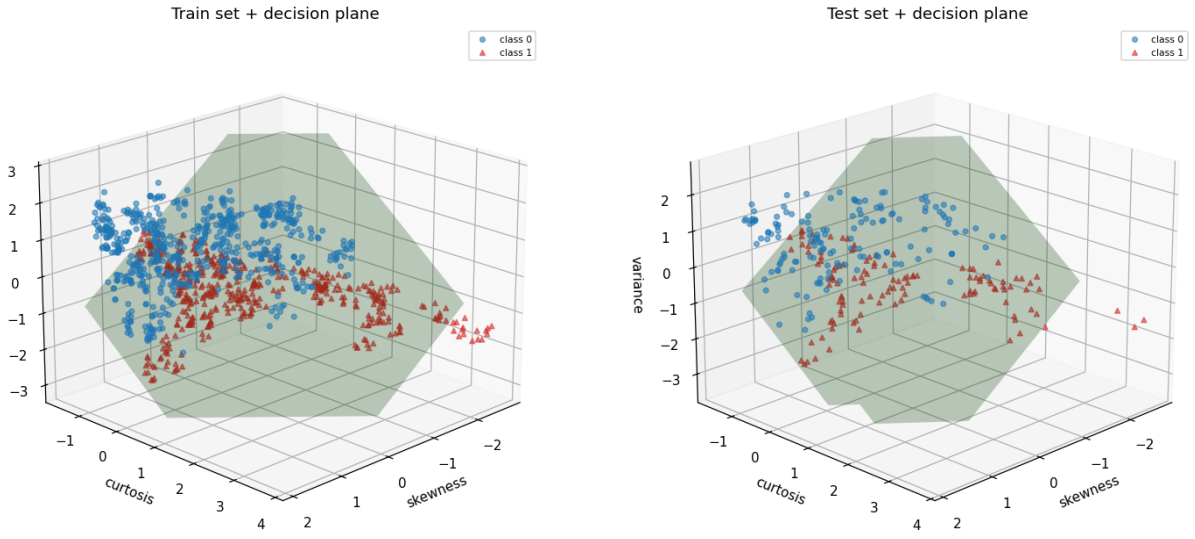


Figure 14: Training set (left) and test set (right) in the three most important standardised features with the separating plane of the logistic-regression model.

Inference: The three features with the largest weights are `skewness` (−8.03), `curtosis` (−7.27) and `variance` (−7.09); `entropy` has a negligible weight (0.12). A model using

only these three features reaches a training accuracy of 0.9891 and a test accuracy of 0.9818, identical to the four-feature model, so `entropy` adds no information. The two classes lie mostly on opposite sides of the plane, and the points near the plane are the ones that are misclassified. The clouds form a slightly curved band rather than two compact blobs, so the classes are almost, but not perfectly, linearly separable.

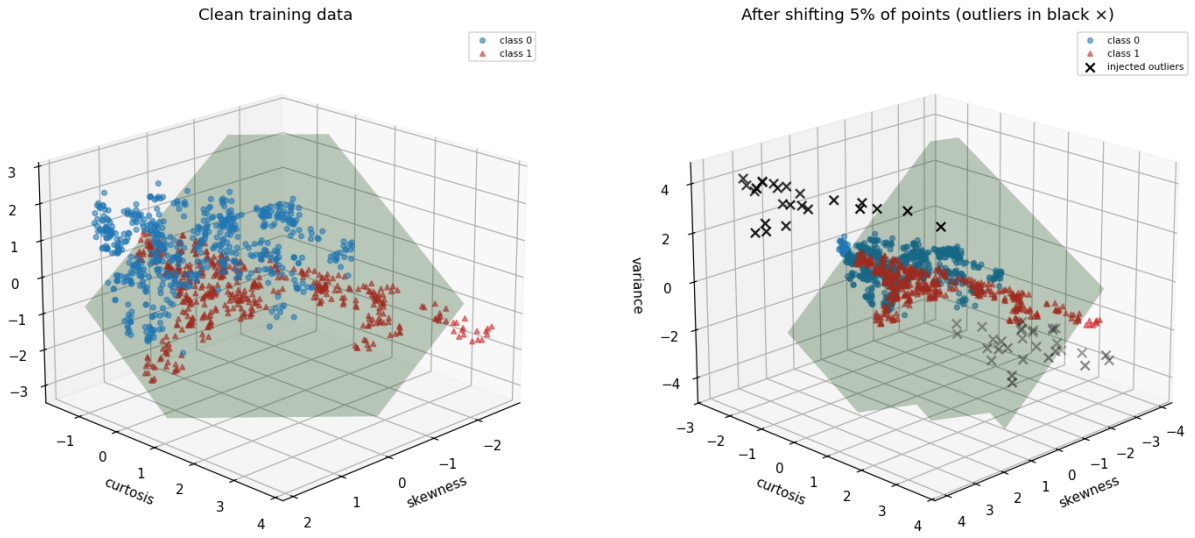


Figure 15: Clean training data (left) and the same data after shifting 5% of the points (right); injected outliers are shown as black crosses.

Inference: In total 55 of the 1097 training points (5%) were shifted by four times the distance between the class means, so they lie far outside the two real clouds in two separate groups. Because the features were re-standardised on the contaminated training data, the standard deviations grew and the genuine data are compressed into a small central region of the plot (compare the axis ranges of the two panels). The outliers therefore distort not only the fitted boundary but also the scaling of every point.

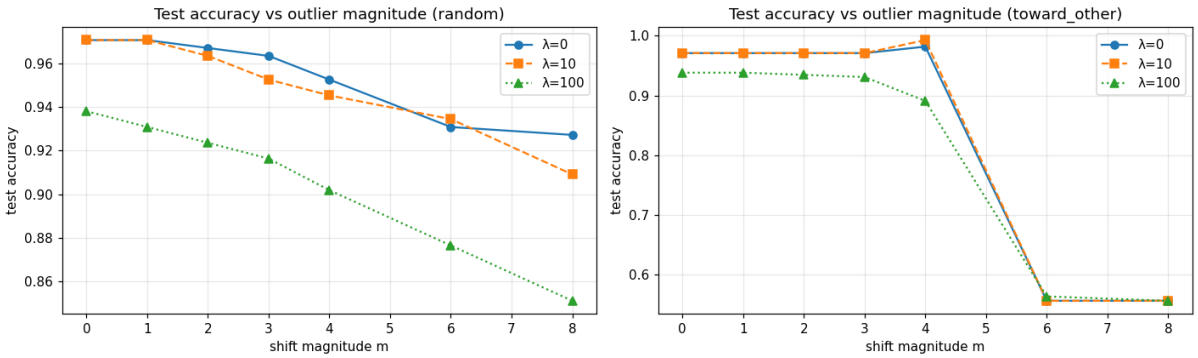


Figure 16: Clean test accuracy versus outlier magnitude m for "random" (left) and "toward_other" (right) contamination, for $\lambda = 0, 10, 100$.

Inference: Random shifts degrade accuracy smoothly: for $m = 8$ the test accuracy falls from 0.971 to about 0.927 ($\lambda = 0$), 0.909 ($\lambda = 10$) and from 0.938 to 0.851 ($\lambda = 100$). For "toward_other" shifts the accuracy is unchanged up to $m = 3$ and is even slightly

higher at $m = 4$, but at $m = 6$ and $m = 8$ it collapses to about 0.555 for all three values of λ, i.e. the majority-class rate. A likely cause is that such extreme outliers inflate the feature standard deviations so much that the standardised genuine data are squeezed together and the model no longer separates them. The strongly regularised model ($\lambda = 100$) is the lowest in the random case, so ℓ_2 regularisation did not protect against outliers here.

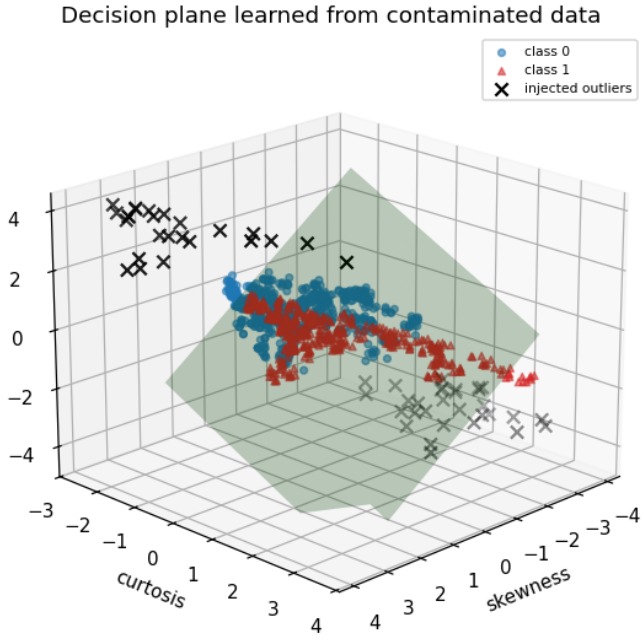


Figure 17: Decision plane learned from the contaminated training data (three features).

Inference: The plane parameters change from $[-2.551, -7.576, -6.776, -6.573]$ on clean data to $[-0.239, -0.664, -0.565, -0.717]$ on contaminated data, roughly ten times smaller, and the four-feature weight norm falls from 12.94 to 1.13. The plane tilts markedly and passes through the dense central region where the genuine data now lie. Its accuracy on the contaminated training set drops to 0.9325, since the injected outliers are by construction on the wrong side, yet this $\lambda = 0$ model still scores 0.9745 on the clean test set (Table 13), so the orientation of the plane still separates the real classes well.

11 Discussion

11.1 Part A – Regression

Observations. The closed-form solution and gradient descent produced identical models (test $R^2 = 0.588$ for OLS, 0.597 for ridge). Ridge regularisation gave a small improvement, and the best λ depended on standardisation (46.4 for standardised, 4.64 for raw data). Without standardisation, gradient descent did not converge in 20 000 iterations and the test R^2 fell to 0.462, whereas it converged in 30 iterations after standardisation. `Ratings`, `ROM`, `RAM` and `Battery_Power` were the main positive drivers of price.

Strengths. The matrix formulation is simple, exact (closed form) and fast, and the weights are easy to interpret once the features are standardised.

Limitations. The test R^2 of about 0.6 and the strong residual pattern show that a linear model is only moderately suitable for this data: the price is extremely skewed and split between a large group of very cheap phones and a smaller group of expensive ones, and the model even predicts negative prices. The data also contain duplicates (32% of the rows), a few implausible screen sizes (44) and, in the test set, few extreme-priced phones, so the reported errors are specific to this split.

Improvements. Predicting log(price) (which removes negative predictions and tames the skew), adding the brand or phone category if available, using interaction or polynomial features, fitting separate models for the cheap and expensive groups, or using non-linear models such as decision-tree ensembles; repeated cross-validation would give a more reliable error estimate than a single 110-sample test set.

11.2 Part B – Classification

Observations. A linear classifier separates the two note classes almost perfectly (test accuracy 0.9818, recall 1.0 for Class 1). Regularisation did not improve accuracy on clean data; it only shrank the weights ($\|w\|$ from 12.9 at $\lambda = 0$ to 4.1 at $\lambda = 10$), and very large λ caused under-fitting down to the majority-class rate. `skewness`, `curtosis` and `variance` are the informative features; `entropy` is not.

Effect of outliers. Contamination changed the learned plane drastically (weights about ten times smaller), but at $m = 4$ the clean-test accuracy of the weakly regularised models did not fall (it changed by -0.0073, $+0.0073$ and $+0.0218$ for $\lambda = 0$, 1 and 10; one test sample corresponds to 0.0036, so these changes are small and partly due to the re-standardisation, not evidence that outliers help). The strongly regularised model ($\lambda = 100$) lost 6.6 percentage points, and for $m \geq 6$ with "toward_other" shifts all models collapsed to about 0.555. Hence ℓ_2 regularisation alone is not a defence against outliers in this experiment.

Strengths and limitations. Logistic regression is fast, convex and interpretable, but it can only draw linear boundaries, and both its weights and the feature scaling are sensitive to extreme points; the accuracies come from one train/test split of 275 test notes.

Improvements. Detect and remove outliers before fitting, use robust scaling (median/IQR) or a robust loss (Huber, ℓ_1), choose λ by cross-validation, report ROC-AUC and use repeated cross-validation, and compare with a non-linear classifier (e.g. a multi-layer network) to check whether the small remaining error is due to non-linearity.

12 Conclusion

Part A. A matrix-based linear regression, solved in closed form and by gradient descent, reached a test R^2 of 0.588 (OLS) and 0.597 (ridge, $\lambda = 46.4$) with an RMSE of about 11 000 on the mobile price data. Standardisation is essential for gradient descent and for a meaningful ridge penalty, and `Ratings`, `ROM`, `RAM` and `Battery_Power` drive the price.

Because of the skewed, two-group price distribution, a linear model is only partly suitable, and a transformed target or non-linear model is recommended.

Part B. A single-layer network (logistic regression) classified bank notes with 98.2% test accuracy and no missed Class 1 notes; the classes are almost linearly separable in `skewness`, `curtosis` and `variance`. ℓ_2 regularisation neither helped nor hurt on clean data until λ became large, and it did not protect the classifier from injected outliers, which can shift the decision plane substantially or, for very large shifts, destroy the model.

13 References

References

[1] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY, USA: Springer, 2009.

[2] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[4] D. Dua and C. Graff, "UCI Machine Learning Repository – Banknote Authentication Data Set," University of California, Irvine, 2019. [Online]. Available: https://archive.ics.uci.edu/dataset/267/banknote+authentication